

Relatório de Segurança e Análise de Vulnerabilidades

— Tech Challenge (Fase 1)

Metadados

Campo	Valor
Projeto	Sistema Integrado de Atendimento e Execução de Serviços — Oficina Mecânica
Repositório	TechChalleng (back-end monolítico Go)
Fase	Fase 1 — Entrega Tech Challenge
Data da análise inicial	22/08/2026
Data da reanálise (pós-correção)	22/08/2026
Responsáveis	Gustavo André Richter, Diego Poletto, Guilherme Montipó Nodari
Ambiente analisado	Desenvolvimento local (Windows 10, Go 1.26.2, Docker Desktop)

1. Metodologia

A análise de segurança seguiu a abordagem **DevSecOps Shift-Left**, aplicando ferramentas automatizadas sobre o código-fonte, dependências e artefato de container **antes** do deploy.

1.1 Ferramentas executadas

Categoria	Ferramenta	Escopo	Objetivo
SAST	gosec v2.28	Código Go (./...)	Detectar padrões inseguros no código (credenciais, HTTP, overflow, logs)
SCA / Dependências	govulncheck v1.7	Módulos Go + stdlib	Rastrear CVEs conhecidas em bibliotecas importadas
Container	Docker Scout / Snyk Container	Imagem techchalleng-oficina:latest	Vulnerabilidades em camadas base (Alpine, ca-certificates)

1.2 Comandos utilizados

```
gosec ./...
govulncheck ./...
docker build -t techchalleng-oficina:latest .
docker scout cves techchalleng-oficina:latest
```

1.3 Controles já presentes no projeto

- Queries SQL parametrizadas via `pgx` (`$1` , `$2` , ...) — mitiga SQL Injection na camada de persistência.
- Value Objects de domínio (`CPF` , `CNPJ` , `Placa` , `Email`) com validação antes da persistência.
- Autenticação JWT em rotas administrativas (`middleware.JWTAuth`).
- Build multi-stage Docker com binário estático (`CGO_ENABLED=0`).

1.4 Controles implementados durante esta análise

- Middleware `InjectionGuard` — bloqueio de padrões SQL/Command Injection em query strings e parâmetros de rota.
- Funções `SanitizeDocumento` e `SanitizePlaca` no pacote `pkg/validator` .
- Timeouts HTTP no `http.Server` (`ReadHeaderTimeout` , `ReadTimeout` , `WriteTimeout`) — mitiga Slowloris (G112).
- Usuário dedicado `appuser` no container de runtime (princípio do menor privilégio).
- **Atualização de dependências vulneráveis** (`pgx` , `jwt` , `x/text` , `x/net`) — ver seção 2.

2. Quadro de Vulnerabilidades

ID	Severidade	Status	Descrição da Brecha	Arquivo / Linha
TC-SEC-001	Alta	 Pendente (dev)	Segredo JWT e credenciais admin com valores padrão hardcoded quando variáveis de ambiente não são definidas (oficina-secret-key-change-in-production, admin123). Risco de token forgery e acesso não autorizado em ambientes expostos.	internal/application/usecase/crud_use 28, 48-50
TC-SEC-002	Média	 Corrigido	Dependência <code>github.com/jackc/pgx/v5@v5.7.1</code> com vulnerabilidade GO-2026-5004 (SQL Injection via placeholder confusion).	go.mod / internal/infra/db/repositorie
TC-SEC-003	Média	 Corrigido	Dependência <code>github.com/golang-jwt/jwt/v5@v5.2.1</code> com GO-2025-3553 (DoS via parsing de header JWT).	go.mod / crud_usecases.go:81
TC-SEC-004	Média	 Corrigido	Servidor HTTP sem <code>ReadHeaderTimeout</code> — vulnerável a Slowloris (gosec G112 , CWE-400).	cmd/api/main.go

TC-SEC-005	Baixa	⚠ Aceito	Conversões int → byte/rune no validador CPF/CNPJ (gosec G115 , CWE-190). Impacto prático nulo (dígitos 0–9).	pkg/validator/validator.go:25-26, 56-5
TC-SEC-006	Baixa	⌚ Parcial	Stdlib Go 1.26.2 com CVEs transitivas (crypto/tls, net/http, html/template, etc.).	Toolchain Go / cmd/api/main.go
TC-SEC-007	Informativa	⌚ Pendente	Swagger UI exposto em /swagger/*any SEM autenticação.	internal/infra/http/handler/router.gc
TC-SEC-008	Média	✅ Corrigido	Dependências golang.org/x/text@v0.19.0 (GO-2026-5970) e golang.org/x/net@v0.30.0 (GO-2025-3595).	go.mod

Legenda de status: ✅ Corrigido · ⌚ Pendente/Parcial · ⚠ Aceito com justificativa

3. Resultados consolidados dos scans

3.1 gosec (SAST) — pós-correção

Summary:
Files : 25
Lines : 4449
Issues : 7 (6× G115 HIGH, 1× G706 LOW)

Comparativo	Antes	Depois
Total de issues	8	7

G112 (Slowloris)	1 (MEDIUM)	0 — eliminado
G115 (overflow validador)	6 (HIGH)	6 (aceitos)
G706 (log injection)	1 (LOW)	1 (baixo risco)

3.2 govulncheck (SCA) — pós-correção

Your code is affected by 11 vulnerabilities from the Go standard library.
(vs. 15 vulnerabilidades em 4 módulos + stdlib antes da correção)

Módulo	Vulnerabilidade	Versão anterior	Versão aplicada	Status
github.com/jackc/pgx/v5	GO-2026-5004	v5.7.1	v5.9.2	✅ Corrigido
github.com/golang-jwt/jwt/v5	GO-2025-3553	v5.2.1	v5.2.2	✅ Corrigido
golang.org/x/text	GO-2026-5970	v0.19.0	v0.41.0	✅ Corrigido
golang.org/x/net	GO-2025-3595	v0.30.0	v0.58.0	✅ Corrigido
Go stdlib	GO-2026-6090, GO-2026-6089, ...	go1.26.2	go1.26.6 (alvo)	⌚ Pendente upgrade toolchain

Comando de atualização executado:

```
go get github.com/jackc/pgx/v5@v5.9.2
go get github.com/golang-jwt/jwt/v5@v5.2.2
go get golang.org/x/text@latest golang.org/x/net@latest
go mod tidy
govulncheck ./...
go test ./...
```

Todos os testes passaram após as atualizações.

4. Análise de Segurança da Infraestrutura Docker

4.1 Dockerfile (multi-stage)

```
# Build stage — golang:1.26-alpine (atualizado)
# Runtime stage — alpine:3.20, usuário appuser (não-root)
```

Controle	Status	Detalhe
Multi-stage build	✅	Reduz superfície de ataque — imagem final contém apenas o binário

Binário estático	✓	CGO_ENABLED=0, flags -ldflags="-w -s"
Usuário não-root	✓	appuser:appgroup — princípio do menor privilégio
Imagens base mínimas	✓	Alpine Linux (~5 MB base)
Go toolchain atualizado	✓	golang:1.26-alpine no stage de build
Secrets no build	✓	Nenhum segredo copiado para camadas da imagem
Healthcheck	⚠	Recomendado adicionar HEALTHCHECK apontando para /health

4.2 docker-compose.yml (desenvolvimento)

- Credenciais explícitas (`oficina_secret` , `admin123`) — **aceitável apenas em ambiente local**.
- Rede isolada `oficina-net` — containers não expostos além das portas mapeadas.

4.3 Recomendações adicionais para produção

1. Usar **Docker secrets** ou variáveis do orchestrator (Kubernetes Secrets, ECS task definitions).
2. Habilitar **TLS termination** no reverse proxy (Nginx/Traefik) com certificados válidos.
3. Executar `docker scout cves` ou `snky container test` em pipeline CI a cada build.
4. Aplicar **read-only filesystem** no container (`read_only: true` no Compose/K8s).

5. Melhorias de segurança implementadas no código

5.1 Middleware InjectionGuard

Arquivo: `internal/infra/http/middleware/security.go`

```
// InjectionGuard bloqueia padrões típicos de SQL Injection e Command Injection
// em query strings e parâmetros de rota antes de chegar aos handlers.
func InjectionGuard() gin.HandlerFunc { ... }
```

Registrado globalmente em `RegisterRoutes` antes dos handlers.

5.2 Sanitização de entradas (CPF, CNPJ, Placa)

Arquivo: `pkg/validator/validator.go`

- `SanitizeDocumento()` — extrai apenas dígitos numéricos.
- `SanitizePlaca()` — permite somente `[A-Z0-9-]`.
- Value Objects em `internal/domain/valobj/documentos.go` aplicam validação algorítmica.

5.3 Hardening HTTP Server

Arquivo: `cmd/api/main.go`

```
srv := &http.Server{
    Addr:           ":" + port,
    Handler:        r,
    ReadHeaderTimeout: 10 * time.Second,
    ReadTimeout:    30 * time.Second,
```

```
WriteTimeout:    30 * time.Second,  
IdleTimeout:     60 * time.Second,  
}
```

6. Plano de ação pós-entrega

Prioridade	Ação	Status
P0	Atualizar pgx, jwt, x/text, x/net	✅ Concluído
P0	Configurar secrets de produção (sem defaults)	⌚ Antes do deploy
P1	Upgrade Go toolchain para 1.26.6+	⌚ Próximo ciclo
P1	Proteger ou remover Swagger em produção	⌚ Próximo ciclo
P2	Adicionar HEALTHCHECK no Dockerfile	Backlog
P2	Integrar scans SAST no CI (GitHub Actions)	Backlog

7. Conclusão

Após a **reanálise pós-correção**, o projeto eliminou **4 vulnerabilidades de dependências externas** (pgx, jwt, x/text, x/net) e **1 vulnerabilidade de configuração HTTP** (Slowloris). Restam **11 CVEs da stdlib Go**, mitigáveis com upgrade para Go 1.26.6+, e **1 achado de alta severidade operacional** (segredos padrão em dev — TC-SEC-001), esperado para ambiente local.

Nenhum achado **crítico** de SQL Injection direto permanece no código graças a queries parametrizadas, Value Objects e middleware `InjectionGuard`.

As mitigações implementadas demonstram postura **DevSecOps** alinhada aos requisitos da Fase 1 do Tech Challenge.